

strapdown-rs: Strapdown inertial navigation and GNSS-degradation simulation in Rust

James Brodovsky ¹✉

¹ Temple University, United States ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Kyle Niemeyer](#) 

Reviewers:

- [@nitishsanghi](#)
- [@abougouffa](#)

Submitted: 05 June 2025

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

An inertial navigation system (INS) estimates position, velocity and attitude by integrating the specific force and angular rate measured by an inertial measurement unit (IMU). Modern IMUs, from the micro-electro-mechanical (MEMS) parts in phones and drones to fibre-optic and ring-laser gyroscopes, are *strapdown*: fixed to the vehicle, with the navigation equations doing the work a gimbaled platform once did. Every IMU drifts, so an INS is normally aided by GNSS. Losing GNSS to jamming, spoofing or obstruction is a central problem in positioning, navigation and timing (PNT) research.

strapdown-rs is a Rust library and command-line toolkit for that research. The strapdown-core crate implements local-level strapdown mechanization and four interchangeable loosely coupled navigation filters. The strapdown-sim binary replays recorded or synthetic IMU/GNSS data through those filters. It withholds, thins or corrupts the GNSS stream according to a seeded scenario described in a configuration file. An experimental strapdown-geonav crate adds gravity- and magnetic-anomaly map matching as a GNSS alternative.

Statement of need

Navigation algorithms are usually prototyped in MATLAB or Python. Prototypes in those languages run fast only when the mathematics is rewritten in vectorized array form. That rewrite is particularly awkward for particle filters, whose natural expression is a loop over particles that each carry their own state. It obscures the algorithm, duplicates logic and is hard to debug. C and C++ give speed, but without memory safety, which makes a library hard to extend safely. strapdown-rs lets filters be written as plainly as the textbook presents them and compiled. The same code can then be reused unchanged from a research script, a CLI experiment or an application.

The second need is reproducible GNSS-denied experiments. Collecting real data under denial requires jamming hardware, test ranges and regulatory approval, so few researchers can, and no field test can be repeated exactly. A simulation can take real or synthetic trajectories and remove or corrupt GNSS in a controlled, seeded way. That lets algorithms be compared under identical conditions, and lets others re-run the comparison. strapdown-rs targets graduate students, researchers and engineers working on INS/GNSS integration and alternative PNT.

State of the field

- gnss-ins-sim ([Aceinna, Inc., 2019](#)) generates synthetic IMU and GNSS data in Python. It has no GNSS fault models, and its filters are examples rather than a reusable library.
- GTSAM ([Dellaert, 2012](#)) is a general factor-graph library with IMU preintegration. Using it for a conventional loosely coupled INS means building the navigation stack yourself, and it offers no degradation scenarios.
- OpenVINS ([Geneva et al., 2020](#)) is a mature visual-inertial estimator built on ROS. It is designed around camera-IMU fusion rather than GNSS-aided navigation.

- Academic MATLAB implementations, typically following Groves (2013), are widely shared. They require a proprietary runtime and are seldom packaged for reuse.

We built a new package rather than contributing to one of these because our two goals are joint. We need a memory-safe compiled library and a seeded scenario engine that share one filter interface. That combination does not fit inside a Python simulator, a C++ factor-graph framework or a ROS stack without rewriting the host project.

Software design

One filter interface. Every filter implements the object-safe `NavigationFilter` trait: predict on an IMU sample, and update against any `MeasurementModel`. The CLI, the `InsEngine` builder for library users, and the test suite all drive filters through this trait. Four implementations share one mechanization, which follows Groves (2013) (Chapter 5.4):

- an error-state Kalman filter (ESKF) with a multiplicative attitude error;
- a total-state extended Kalman filter (EKF);
- an unscented Kalman filter (UKF);
- the Rao-Blackwellized particle filter of Canciani & Raquet (2017). It samples only horizontal position and shares one Kalman filter across all particles for the remaining states.

The ESKF is the default. Its error state stays small and near zero, which keeps linearization accurate and avoids attitude-parameterization singularities. It estimates IMU biases at modest cost, and on the regression scenarios its accuracy is indistinguishable from the UKF's. The EKF's analytic Jacobians are tested against finite differences of the mechanization, because a wrong Jacobian does not crash a filter; it quietly degrades it.

Scenarios as an event stream. Input records become a time-ordered stream of IMU propagation steps and measurement events. Two independent stages act on that stream: - *Scheduling* decides when GNSS is available: pass-through, fixed-interval, or duty-cycle outages. - *Fault injection* corrupts the fixes that are delivered: correlated AR(1) noise, slow bias drift, position hijacking, or combinations of these.

A filter therefore cannot distinguish a simulated outage from a real one, and new faults require no filter changes. All randomness is seeded.

Correctness over convenience. - The library returns a `StrapdownError` instead of panicking, and lints deny `unwrap`, `expect` and `panic` in library code. - The navigation frame (NED by default, ENU on request) is checked against the gravity direction measured in the data rather than assumed. - Process noise is specified as a spectral density, so tuning does not change with sample rate.

Functionality

- Strapdown mechanization on the WGS84 ellipsoid (National Geospatial-Intelligence Agency, 2014) with Somigliana gravity, built on nav-types (Nordmoen & Feuerstein, 2022).
- Aiding by:
 - GNSS position and velocity;
 - barometric altitude, with bias estimation;
 - magnetometer heading, corrected for declination by the World Magnetic Model (NOAA NCEI Geomagnetic Modeling Team and British Geological Survey, 2024);
 - zero-velocity and zero-angular-rate updates, triggered by a stationarity detector.
- Chi-squared innovation gating with recovery.
- Coarse alignment, IMU calibration and lever-arm compensation.
- GNSS degradation scenarios configured from TOML, YAML or JSON files or from CLI flags.

- A synthetic trajectory generator with IMU error models by sensor grade.
- Accuracy and consistency metrics: RMSE, CEP, NEES and NIS.
- CSV output from the CLI, plus HDF5, NetCDF and MCAP writers in the library.
- Experimental geophysical navigation in strapdown-geonav.
- A Python analysis package for preprocessing and statistics.

The documentation is an mdBook user guide plus API documentation. A lint makes missing_docs an error, so every public item is documented.

Research impact statement

strapdown-rs is the experimental platform behind two conference papers on navigating MEMS-grade platforms without GNSS using gravity and magnetic anomaly maps. One uses a UKF (Brodovsky & Dames, 2026b) and the other a particle filter (Brodovsky & Dames, 2026a). Both are built on the toolkit's filters, its GNSS-denial scenarios and its geophysical measurement models. The scenario recipes for these studies are versioned in the repository's conf/ directory.

The reproducibility claims are tested rather than asserted: - About 820 Rust tests run in continuous integration on Linux, macOS and Windows, alongside 57 Python tests. - An accuracy regression suite runs every filter over recorded and synthetic scenarios: clean GNSS, sparse fixes, a 60-second outage, degraded GNSS, and dead reckoning. - The suite fails the build when a gated metric moves beyond its recorded tolerance. The baseline tables in the documentation are generated from the same results. - Synthetic trajectories make every scenario reproducible without the original recordings.

The v1.0 release publishes strapdown-core, strapdown-sim and strapdown-geonav to crates.io. Other Rust projects can then depend directly on the mechanization and filters, and compare new estimators against the provided baselines under identical scenarios.

AI usage disclosure

Generative AI was used substantially in this project. The repository history records commits authored by Anthropic's Claude Code and by the GitHub Copilot coding agent alongside the author's own. These tools implemented features, wrote tests and documentation, and investigated defects. The author directed the work, reviewed each change before merging it, and checked the algorithms against Groves (2013) and Canciani & Raquet (2017).

Correctness rests on checks that do not depend on trusting the generated code: - the unit and integration tests; - finite-difference verification of the analytic Jacobians; - the accuracy baseline gated in CI; - lints that forbid panics in library code.

This paper was drafted with AI assistance from the author's notes and the code, then revised and verified by the author.

Acknowledgements

This work received no external funding.

References

- Aceinna, Inc. (2019). *Gnss-ins-sim: GNSS/INS simulator for testing and prototyping navigation algorithms*. <https://github.com/Aceinna/gnss-ins-sim>
- Brodovsky, J., & Dames, P. (2026a). Navigation in GNSS-denied environments using MEMS-grade sensors and geophysical anomalies: A particle filter approach. *Proceedings of the ION 2026 Pacific PNT Meeting*, 399–410. <https://doi.org/10.33012/2026.20618>
- Brodovsky, J., & Dames, P. (2026b). Navigation in GNSS-denied environments using MEMS-grade sensors and geophysical anomalies: A UKF approach. *Proceedings of the 2026*

- 134 *International Technical Meeting of the Institute of Navigation*, 155–164. [https://doi.org/](https://doi.org/10.33012/2026.20508)
135 [10.33012/2026.20508](https://doi.org/10.33012/2026.20508)
- 136 Canciani, A., & Raquet, J. (2017). Airborne magnetic anomaly navigation. *IEEE Transactions*
137 *on Aerospace and Electronic Systems*, 53(1), 67–80. [https://doi.org/10.1109/TAES.2017.](https://doi.org/10.1109/TAES.2017.2649238)
138 [2649238](https://doi.org/10.1109/TAES.2017.2649238)
- 139 Dellaert, F. (2012). *Factor graphs and GTSAM: A hands-on introduction* (GT-RIM-CP&R-
140 2012-002). Georgia Institute of Technology. <https://gtsam.org>
- 141 Geneva, P., Eckenhoff, K., Lee, W., Yang, Y., & Huang, G. (2020). OpenVINS: A research
142 platform for visual-inertial estimation. *Proceedings of the IEEE International Conference*
143 *on Robotics and Automation (ICRA)*, 4666–4672. [https://doi.org/10.1109/ICRA40945.](https://doi.org/10.1109/ICRA40945.2020.9196524)
144 [2020.9196524](https://doi.org/10.1109/ICRA40945.2020.9196524)
- 145 Groves, P. D. (2013). *Principles of GNSS, inertial, and multisensor integrated navigation*
146 *systems* (2nd ed.). Artech House. ISBN: 978-1-60807-005-3
- 147 National Geospatial-Intelligence Agency. (2014). *Department of defense world*
148 *geodetic system 1984: Its definition and relationships with local geodetic systems*
149 (No. NGA.STND.0036_1.0.0_WGS84). National Geospatial-Intelligence Agency.
150 <https://earth-info.nga.mil/>
- 151 NOAA NCEI Geomagnetic Modeling Team and British Geological Survey. (2024). *World*
152 *magnetic model 2025*. NOAA National Centers for Environmental Information. [https:](https://doi.org/10.25921/aqfd-sd83)
153 [//doi.org/10.25921/aqfd-sd83](https://doi.org/10.25921/aqfd-sd83)
- 154 Nordmoen, J., & Feuerstein, M. (2022). *Nav-types: Easy to use navigation types for Rust*.
155 <https://github.com/nordmoen/nav-types>